

```

#!/usr/bin/env node

const https = require('https');
const nodemailer = require('nodemailer');

// Get environment variables
const ANTHROPIC_API_KEY = process.env.ANTHROPIC_API_KEY;
const EMAIL_USER = process.env.EMAIL_USER;
const EMAIL_PASSWORD = process.env.EMAIL_PASSWORD;
const RECIPIENT_EMAIL = process.env.RECIPIENT_EMAIL;

if (!ANTHROPIC_API_KEY || !EMAIL_USER || !EMAIL_PASSWORD || !RECIPIENT_EMAIL) {
  console.error('Missing environment variables. Set: ANTHROPIC_API_KEY, EMAIL_USER, EMAIL_PASSWORD, RECIPIENT_EMAIL');
  process.exit(1);
}

async function callClaudeAPI(prompt) {
  return new Promise((resolve, reject) => {
    const requestBody = JSON.stringify({
      model: 'claude-opus-4-6',
      max_tokens: 4000,
      messages: [
        {
          role: 'user',
          content: prompt
        }
      ]
    });

    const options = {
      hostname: 'api.anthropic.com',
      path: '/v1/messages',
      method: 'POST',
      headers: {
        'x-api-key': ANTHROPIC_API_KEY,
        'anthropic-version': '2023-06-01',
        'content-type': 'application/json',
        'content-length': Buffer.byteLength(requestBody)
      }
    };

    const req = https.request(options, (res) => {
      let data = '';
      res.on('data', (chunk) => { data += chunk; });
    });
  });
}

```

```

    res.on('end', () => {
      try {
        const response = JSON.parse(data);
        if (response.content && response.content[0] && response.content[0].text)
          resolve(response.content[0].text);
        } else {
          reject(new Error('Unexpected API response format'));
        }
      } catch (e) {
        reject(e);
      }
    });
  });
});

req.on('error', reject);
req.write(requestBody);
req.end();
});
}

```

```

async function sendEmail(htmlContent) {
  const transporter = nodemailer.createTransport({
    service: 'gmail',
    auth: {
      user: EMAIL_USER,
      pass: EMAIL_PASSWORD // Use Gmail App Password, not regular password
    }
  });
}

```

```

const today = new Date().toISOString().split('T')[0];

```

```

const mailOptions = {
  from: EMAIL_USER,
  to: RECIPIENT_EMAIL,
  subject: `Daily News Digest - ${today}`,
  html: htmlContent
};

```

```

return transporter.sendMail(mailOptions);
}

```

```

async function main() {
  try {
    console.log('🚀 Generating daily news digest...');

    const prompt = `Scan the web for the latest 24-48 hours of news and create a

```

Structure the email in THREE SECTIONS:

****SECTION 1: India Startup & VC News****

Include 4-6 stories covering: funding rounds, IPOs, acquisitions, earnings, and f
For each story: headline + 2-3 sentence summary + "why it matters" callout.
End this section with ONE finance term (related to startups/VC) explained in plai

****SECTION 2: India Macro News****

Include 4-6 stories covering: RBI decisions, SEBI updates, government policy, tra
For each story: headline + 2-3 sentence summary + "why it matters" callout.
End this section with ONE finance term (related to macro/policy) explained in pla

****SECTION 3: Global Macro News****

Include 4-6 stories covering: US Fed actions, global VC trends, geopolitics, and
For each story: headline + 2-3 sentence summary + "why it matters" callout.
End this section with ONE finance term (related to global macro) explained in pla

****CLOSE WITH: "Pattern of the Day"****

Write ONE insight that connects all three sections together. Show how India start

Format as clean HTML (no fancy styling, just readable structure with clear header

```
const htmlContent = await callClaudeAPI(prompt);

console.log('✅ Digest generated. Sending email...');

await sendEmail(htmlContent);

console.log('📧 Email sent successfully!');
} catch (error) {
  console.error('❌ Error:', error.message);
  process.exit(1);
}
}

main();
```